CSC 335 — Midterm Study Guide

Banner — scope PROVISIONAL. Midterm scope is **not published**. Practice covers the lectures so far plus the syllabus weeks 1–6 topics. The **date is set**: Thu Oct 8, 2026, in the class period; the clock time is TBD beyond "class period".

No Gradescope. CSC 335 is D2L-primary, and the Classroom50 FA26 page returns 404. **No Gradescope format is used or invented here.** On-box decks may show **2025** headers; they are used for FA26 topic practice only.

P3 is Needs verification: it is in the syllabus but not in D2L Assignments.

Canonical maps: `csc335-topic-map.md` · `announced-assessments-inventory-2026-09-24.md`

1. Header

Field	Value	Status
Course	CSC 335 Object-Oriented Programming and Design (FA26), TuTh 12:30–1:45 PM, The Commons 305	Confirmed
Assessment	Midterm Exam	Confirmed
Date	**Thu Oct 8, 2026**	**Scheduled** (D2L / syllabus)
Time	Class period; exact clock time **TBD**	NV
Format / materials	**Not published.** Closed-book and length are not in the inventory.	**Verify** on D2L or in class
Coverage	Decks on box (Intro, OOP, Classes 1–2, Testing & Javadoc) plus syllabus weeks 3–6 topics	**PROVISIONAL**
Same day	CSC 345 Exam 1 is also Oct 8. If both are held in class, they are **back-to-back in the same room**: 335 at 12:30, 345 at 2:00, both in Commons 305. Clock times are unverified.	Plan energy
Guide updated	2026-09-24 (America/Phoenix) — Opus review pass	

Sources

Source	Role
`csc335-topic-map.md` + inventory	Syllabus weeks 1–7, date, gaps
`csc335-intro.pdf`, `csc335-oop.pdf`, `csc335-classes-part1.pdf`, `csc335-classes-part2.pdf`, `csc335-testing-and-javadoc.pdf` (on box)	Lecture content (headers may show 2025)
`csc335-fall2026-syllabus.pdf`	Midterm date and weekly topics

Source-status legend. Each topic is tagged with one of these:

- **[Deck]** — a deck is on box.
- **[Syllabus-only]** — the syllabus lists the topic but no deck was observed. Practice uses standard Java or Gang-of-Four definitions; **match the lecture's notation.**

---

2. Topic checklist

Order	Section ID	Topic	Source	Blind ✓	Check ✓	Retake ✓
1	`335-M-intro`	Course intro, OOP motivation, UML basics	[Deck] wk 1			
2	`335-M-oop`	Encapsulation, inheritance, polymorphism, cohesion and coupling	[Deck] wk 1			
3	`335-M-classes1`	Classes, composition, UML, information hiding	[Deck]			
4	`335-M-testing`	Testing (JUnit) and Javadoc	[Deck] wk 2			
5	`335-M-classes2`	Class hierarchies, static vs instance, exceptions	[Deck] wk 3			
6	`335-M-lambdas`	Anonymous classes and lambdas	[Syllabus-only] wk 3			
7	`335-M-generics`	Java generics; frameworks; intro to design patterns	[Syllabus-only] wk 4			
8	`335-M-collections`	Collections Framework; Iterator and Template Method patterns	[Syllabus-only] wk 5			
9	`335-M-javafx-obs`	JavaFX basics; Observer and Composite patterns	[Syllabus-only] wk 6			

Week 7 is "Review for midterm." Nothing new.

---

3. Explanations + worked examples

Intro / OOP (`335-M-intro`, `335-M-oop`) [Deck]

- Programming moved from unstructured, to structured, to object-oriented code.
- **Encapsulation** bundles data with the methods that use it and hides the representation behind a controlled interface.
- **Inheritance** means "is-a": a subclass reuses and extends a superclass.
- **Polymorphism:** one interface, many implementations. The method that runs is chosen by the object's **runtime** type.
- **Cohesion** should be high: a class does one job. **Coupling** should be low: a class depends on few things, preferably through interfaces.

Worked example. A `BankAccount` that only manages its balance is cohesive. If it also renders UI and sends email, cohesion is low. A UI that uses a concrete `MysqlBankStore` everywhere is tightly coupled; depending on a `BankStore` interface is loosely coupled.

Classes 1 (`335-M-classes1`) [Deck]

- **Composition ("has-a")** means the part's lifetime is owned by the whole. In UML it is a **filled diamond on the whole's side**.
- **Aggregation** is a weaker has-a, drawn with a hollow diamond. **Association** is a plain line. **Inheritance** is a hollow triangle pointing to the superclass. **Interface realization** is a dashed line with a hollow triangle.
- **UML class box:** the name, then fields, then methods. `-` means private, `+` public, `#` protected.
- **Information hiding:** fields are private, with accessors only where needed, and no leaking of mutable internals.

Testing & Javadoc (`335-M-testing`) [Deck]

- **Unit tests** check one class or method in isolation. **Integration tests** check parts working together.
- **Black-box** tests come from the specification. **White-box** tests come from the code's paths and branches.
- **JUnit:** a fixture method runs before each test, `@Test` marks tests, and assertions check results. Tests should be independent of each other and of their order.

	JUnit 4 (deck style)	JUnit 5
Before each test	`@Before`	`@BeforeEach`
After each test	`@After`	`@AfterEach`
Expecting an exception	`@Test(expected = X.class)`	`assertThrows(X.class, () -> ...)`

- Java `assert` statements are **disabled by default**; enable them with `-ea`. JUnit assertions are separate from `assert`.
- **Javadoc** uses `/** ... */` with `@param`, `@return`, and `@throws`. Document every public class and method.

Classes 2 (`335-M-classes2`) [Deck]

- **Static** members belong to the class and are shared by all instances. **Instance** members belong to each object.
- **Overriding** replaces an inherited method with the same signature, chosen at runtime. **Overloading** means same name, different parameters, chosen at compile time.
- `super(...)` calls the parent constructor and must come first in the constructor. `super.m()` calls the parent's version of `m`.
- **Exceptions:**
- Checked exceptions (such as `IOException`) must be caught or declared with `throws`.
- Unchecked exceptions (`RuntimeException` subclasses such as `IllegalArgumentException`) do not need to be.
- A `finally` block always runs, even after a `return`.

Lambdas (`335-M-lambdas`) [Syllabus-only]

- An **anonymous class** implements an interface inline: `new Comparator<String>() { public int compare(...) {...} }`.

- A **lambda** is a shorter form for a **functional interface** (one abstract method): `(a, b) -> a.length() - b.length()`.
- A **method reference** is shorter still: `String::length`, `System.out::println`.
- Lambdas can use local variables only if they are **effectively final**.

Generics (`335-M-generics`) [Syllabus-only]

- `class Box<T> { private T item; ... }` is type-safe reuse, checked at compile time. There is no casting, and wrong types are caught early.
- **Type erasure:** generic type information is removed at runtime, so `new T()` and `List<int>` are not allowed. Use `List<Integer>`, which relies on autoboxing.
- **Bounded types:** `<T extends Comparable<T>>` lets you call `compareTo` on T.
- **Design patterns** are named, reusable solutions to recurring design problems. A **framework** calls your code ("inversion of control"), whereas you call a library.

Collections, Iterator, Template Method (`335-M-collections`) [Syllabus-only]

Collections Framework:

- `List` is ordered and allows duplicates (`ArrayList`, `LinkedList`).
- `Set` has no duplicates (`HashSet`, `TreeSet` sorted).
- `Map` stores key→value pairs (`HashMap`, `TreeMap`).
- `Queue` and `Deque` support FIFO and LIFO use.

`equals` and `hashCode`. If you override `equals`, you must also override `hashCode`, or `HashSet` and `HashMap` will misbehave.

Iterator pattern. Traverse a collection without exposing its internals, using `hasNext()` and `next()`. Removing an element during a for-each loop throws `ConcurrentModificationException`; use `iterator.remove()` instead.

Template Method pattern. A superclass method defines the fixed **skeleton** of an algorithm and calls abstract or "hook" steps that subclasses override. The skeleton is often `final`.

JavaFX, Observer, Composite (`335-M-javafx-obs`) [Syllabus-only]

- **JavaFX structure:** a `Stage` (the window) holds a `Scene`, which holds a tree of `Node`s. Layout panes include `BorderPane`, `VBox`, and `HBox`. Event handlers are often lambdas: `button.setOnAction(e -> ...)`.
- **Observer pattern.** A subject keeps a list of observers and notifies them when it changes. This decouples a model from its views. (Java's old `Observable` class is deprecated; lectures may still show it.)
- **Composite pattern.** Treat individual objects and groups of objects uniformly through a shared interface. A group holds children and forwards operations to them. A JavaFX scene graph is an example: a `Pane` contains `Node`s and is itself a `Node`.

---

4. Practice questions (core)

P1 (recall). Define encapsulation in one sentence.

P2 (explain). Why are high cohesion and low coupling both desirable?

P3 (UML). Sketch UML for a `Library` that is composed of many `Book`s.

P4 (testing). Write a test expecting `deposit(-5)` to throw `IllegalArgumentException`.

P5 (recall). What is the difference between black-box and white-box testing?

P6 (Javadoc). Write Javadoc for `E pop()` on a stack that throws `EmptyStackException` when empty.

P7 (explain). When should a method be an instance method rather than static?

P8 (trace). `Dog extends Animal`, and both define `speak()`. Which version runs for `Animal a = new Dog(); a.speak();`?

P9 (recall). When is the midterm, and what is still unverified?

P10 (explain). Why is P3 not treated as confirmed study material in this guide?

4b. Written-exam practice — generic shapes (not Gradescope-derived)

Where this format comes from. CSC 335 has no Gradescope, and its midterm format is unpublished. These items use generic written-exam shapes: short answer, code tracing, code writing, UML, and tests. **This is a practice recommendation, not an inferred format.**

Timed mock (Mon Oct 5): do T1–T4, W1–W3, and S1–S4 in **50 minutes** on paper.

Code tracing

T1 (static vs instance). What does this print?

```
class Counter {
  static int total = 0;
  int mine = 0;
  void hit() { total++; mine++; }
}
Counter a = new Counter(), b = new Counter();
a.hit(); a.hit(); b.hit();
System.out.println(a.mine + " " + b.mine + " " + Counter.total);
```

T2 (dynamic dispatch through a superclass method). What does this print?

```
class Animal {
  String speak() { return "..."; }
  String intro() { return "I say " + speak(); }
}
class Cat extends Animal {
  @Override String speak() { return "meow"; }
}
Animal x = new Cat();
System.out.println(x.intro());
```

T3 (exceptions and finally). What do `f(-1)` and `f(5)` print?

```
static int f(int x) {
  try {
    if (x < 0) throw new IllegalArgumentException();
    return 1;
  } catch (IllegalArgumentException e) {
    return 2;
  } finally {
    System.out.print("F ");
  }
}
System.out.println(f(-1));
System.out.println(f(5));
```

T4 (overloading vs overriding). What does this print?

```
class P { void m(Object o) { System.out.println("P-Object"); } }
class C extends P {
  void m(Object o) { System.out.println("C-Object"); }
  void m(String s) { System.out.println("C-String"); }
}
P p = new C();
p.m("hi");
```

Code writing

W1 (encapsulation fix). Rewrite this class so the balance can never go negative and cannot be set directly:

```
public class Account { public double balance; }
```

W2 (lambda / comparator). Sort a `List<String> names` by length, then alphabetically for ties, using a lambda or `Comparator` methods.

W3 (generics). Write a generic static method `max` that returns the largest element of a non-empty `List<T>`, where T is `Comparable`.

Short answer

S1. Name the pattern: (a) a spreadsheet cell's charts update whenever the cell changes; (b) a `Folder` contains `File`s and other `Folder`s, and `size()` works on both; (c) a base `Game.play()` fixes the order `setup(); takeTurns(); scoreboard();`, and subclasses fill in the steps.

S2. Why must `hashCode` be overridden whenever `equals` is?

S3. Checked or unchecked: `IOException`, `NullPointerException`, `IllegalArgumentException`, `FileNotFoundException`.

S4. Write three JUnit test *names* (not bodies) for a `Stack<E>` that cover normal use, an edge case, and an error case.

5. ANSWER KEY

Core keys

A1. Bundle data with the methods that use it, and hide the internal representation behind a controlled interface.

A2. Cohesive classes are easier to understand, test, and change. Low coupling limits ripple effects when something changes.

A3. `Library` ◆——— `Book`, with the filled diamond on the Library side and multiplicity `*` (or `0..*`) on the Book end. Follow the Classes 1 deck's notation.

A4.

```
// JUnit 4
@Test(expected = IllegalArgumentException.class)
public void depositNegativeThrows() { acct.deposit(-5); }

// JUnit 5
@Test
void depositNegativeThrows() {
  assertThrows(IllegalArgumentException.class, () -> acct.deposit(-5));
}
```

A5. Black-box tests are derived from the specification and inputs/outputs, without looking at the code. White-box tests use the code's structure, aiming to cover its branches and paths.

A6.

```
/**
 * Removes and returns the top element of this stack.
 *
 * @return the element that was on top
 * @throws EmptyStackException if the stack is empty
 */
public E pop() { ... }
```

A7. When the behavior depends on, or changes, a particular object's state.

A8. `Dog.speak()`. The runtime type decides (dynamic dispatch).

A9. Thu Oct 8, 2026, in the class period. Still unverified: the clock time, format, materials, and published scope.

A10. The syllabus lists P3, but it is missing from D2L Assignments, so it is marked **Needs verification**, and no study time is planned around it.

Written-exam keys

T1. `2 1 3`

T2. `I say meow.` `intro()` is inherited, but `speak()` dispatches on the runtime type, which is `Cat`.

T3. `F 2` then `F 1`. `finally` prints before each returned value reaches `println`.

T4. `C-Object`. Overload resolution happens at **compile time** using the static type `P`, which only has `m(Object)`. At runtime, the override in `C` runs.

W1.

```
public class Account {
  private double balance;

  public double getBalance() { return balance; }

  public void deposit(double amt) {
    if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
    balance += amt;
  }

  public void withdraw(double amt) {
    if (amt <= 0 || amt > balance) throw new IllegalArgumentException("invalid amount");
    balance -= amt;
  }
}
```

W2.

```
names.sort(Comparator.comparing(String::length)
                     .thenComparing(Comparator.naturalOrder()));
```

An equivalent lambda:

```
names.sort((a, b) -> a.length() != b.length()
    ? Integer.compare(a.length(), b.length())
    : a.compareTo(b));
```

W3.

```
public static <T extends Comparable<T>> T max(List<T> xs) {
  T best = xs.get(0);
  for (T x : xs) if (x.compareTo(best) > 0) best = x;
  return best;
}
```

S1. (a) Observer. (b) Composite. (c) Template Method.

S2. Equal objects must have equal hash codes. Otherwise hash-based collections (`HashSet`, `HashMap`) look in the wrong bucket and miss "equal" objects, which produces duplicates or failed lookups.

S3. `IOException`: checked. `NullPointerException`: unchecked. `IllegalArgumentException`: unchecked. `FileNotFoundException`: checked (it is a subclass of `IOException`).

S4. For example: `pushThenPopReturnsSameItem`, `newStackIsEmpty`, `popOnEmptyThrows`.

---

6. Common mistakes + readiness

- Confusing inheritance ("is-a") with composition ("has-a"), or putting the UML diamond on the wrong end.
- Public fields "just this once."
- Tests that depend on the order they run in, or that share mutable state without a before-each fixture.
- Forgetting Javadoc on public API.
- Catching generic `Exception` and swallowing it.
- Confusing overloading (chosen at compile time) with overriding (chosen at runtime). See T4.
- Overriding `equals` without `hashCode`.
- Modifying a collection inside a for-each loop.

Readiness: a blind pass over all nine IDs, one UML redraw, three JUnit tests written, the 50-minute mock, and a light mixed review on Wed Oct 7 — **no cram on Wednesday night**.

---

7. Day-by-day prep

The same day as CSC 345 Exam 1, so split effort. Prefer Mon/Wed/Fri and weekends for 335.

Day	Focus	IDs
Fri Oct 2	Intro, OOP, Classes 1	intro, oop, classes1
Sat Oct 3	Testing, Classes 2, lambdas	testing, classes2, lambdas
Sun Oct 4	Generics, collections, patterns, JavaFX	generics, collections, javafx-obs
Mon Oct 5	**50-minute mock** (§4b), then retake misses	all
Tue Oct 6	Optional light flash (the 337 midterm is today)	—
Wed Oct 7	Light mixed review, not a cram	all, light
Thu Oct 8 morning	Calm buffer only	—

---

8. Changes in this review (2026-09-24)

- **Filled the scope gap.** The banner says "syllabus weeks 1–6", but the old guide covered only the week 1–3 decks. Added `335-M-lambdas`, `335-M-generics`, `335-M-collections`, and `335-M-javafx-obs`. Each is tagged **[Syllabus-only]**, because no deck was observed; they use standard Java and Gang-of-Four definitions.
- Added a JUnit 4 vs JUnit 5 table, since the deck uses `@Before`, and made the P4 key show both styles.
- Added generic written-exam practice (T1–T4, W1–W3, S1–S4) with keys. It is explicitly **not** Gradescope-derived.
- Changed "closed-book" to **verify**, and noted the possible back-to-back schedule with the CSC 345 exam.
- Improved UML notation guidance (composition, aggregation, inheritance, realization).
- Changed times to Arizona time.

9. Disclaimer

CSC 335 is D2L-primary, with no Gradescope course. No Gradescope format is used or invented. Do not start D2L quiz attempts to scrape questions.

Study aid only — not for submitted work. Follow CSC 335 and UA policies.